

The plan from here — what we agreed, and why

For Michael. Written at the end of books-43, after we merged everything to main.

You said something at the end of our last exchange that this whole document exists to answer:

"Please make sure to record everything we have just worked out as the strategy and roadmap going forward for the next several slices. I don't want to miss something and drift from what's important."

That is exactly the right instinct, and it is the single most useful thing you have asked me for. Conversations get compacted. Sessions end. My memory of this discussion will not survive; the repository will. So the plan now lives in two places, on purpose:

- docs/BOOKS_ROADMAP.md — the tracked source of truth. Tables, counts, order. Terse.
- This file — the same plan in plain English, explaining why each piece sits where it sits.

And a third thing, which is the part I want you to notice: there is now a **test** that fails if either document drifts from reality. tests/compliance/books-roadmap-agreed-order.test.ts re-counts every number in both files from the actual code, every time the suite runs. If someone edits the order, paraphrases your instruction, or lets a count go stale, the build goes red and says which line lied. **A roadmap nobody checks is worse than no roadmap, because you would plan around it.**

Then I attacked that test eighteen different ways to prove it actually fires — re-ordering the slices, dropping a module from the table, nudging a count by one, deleting your document outright. All eighteen were caught. That number is only meaningful because of two **controls** that go the other way: I also made two harmless edits — added a sentence, reworded an explanation — and required the test to stay **green**. Without those, a test that panicked at every keystroke would have scored a perfect eighteen out of eighteen and told you nothing.

That idea is worth more than the test is. In your language, it is dual-purpose testing: a control that rejects everything is not a control, it is an obstacle. When someone shows you a system that caught every error you threw at it, the next question is always *"and what did it correctly let through?"* A check with no false-positive test attached is a check nobody has finished evaluating.

I will also tell you the part that went wrong, because it is the more useful half. One of those eighteen attacks reported **"not caught"** — and the test was innocent. My attack script had silently failed to make the edit at all, because of a character-encoding detail in the arrow symbol I was searching for. Nothing was broken; nothing had been tested either. The output looked *identical* to a real hole. I nearly spent an hour fixing a test that was working perfectly.

So the harness now checks that each edit **actually landed** before it judges the result, and reports "no-op" as its own category. The reason that matters to you: **a test that does not run looks exactly like a test that passes.** That is true of reconciliations too. A tie-out script that silently reads an empty file will tell you the accounts agree, and it will be right — nothing disagreed with nothing.

The order we agreed: C → A → D → B

Slice	In one sentence
C	Connect the unreachable teaching
A	Finish the W-2 / W-3 engine
D	The Form / Why / Check tabs
B	K-1, 1120-S, 1040

Why C is first: you are already paying for teaching you cannot see

This is the finding I most want you to understand, because it is a little embarrassing and very instructive.

There are **six teaching modules** in this system — 82 individual lessons, 2,345 lines — that are **fully written, fully tested, and reachable from no screen whatsoever**. Every one has passing tests. Every one is invisible to you.

What it teaches	Lessons
Payroll onboarding (I-9, W-4, SSN handling, first paycheck)	33

Tax penalties	20
Interest under §6621	10
Period close	9
The S-corporation year	5
Payroll reconciliation	5
Total	82

Each lesson has the same five parts, and I want you to see the shape because it is the shape of good teaching:

- **In plain English** — what this thing does, one breath, no jargon
- **Why it exists** — what goes wrong in the real world without it
- **The trap** — *the specific expensive mistake a competent person actually makes here*
- **What I would do** — what a CPA in your chair would actually do
- **The authority** — the statute or regulation behind it, quoted word for word

That third one is the money. Not "here is how the button works" but "here is the mistake you are personally likely to make, and here is what it costs."

Why this happened, honestly: the tests only ever proved the lessons were *well-formed*. No test could prove you had a way to *read* them, because you did not. The repo has a name for this — *dead code wearing a green check* — and it is precisely the failure mode we keep catching in the numbers. It turns out it applies to teaching too.

So C is first because it is the cheapest real gain available. Nothing has to be invented. It only has to be **connected**.

One technical wrinkle I found while measuring, so it does not surprise either of us later: four of the six modules read files from disk at load time — a build-time self-check that makes sure every function has a lesson. That kind of code cannot run in a web browser. So the wiring has to pull the *lesson text* across without dragging the *file reading* behind it. There is already a pattern in this repo for exactly that (`ytd-mentor` does it), so we follow it rather than invent something. I mention it because "just import it" would have failed, and I would rather you know I checked than watch me discover it later.

Two smaller items ride along with C:

- **Nine broken citations.** Washington statute quotes that point at files which either were never downloaded (five) or are being looked for under the wrong name (four — the code truncates `50A.10.030` to `50`, the same bug that once sent §280E looking for `usc-280`). These have been showing up as a red line in every verification run for many slices. That is corrosive: **a permanent red line trains everybody to ignore red lines.** Order matters here — fix the name-finding bug *before* downloading, or the text lands under a filename nothing looks for.
- **One duplicated rounding function**, where one copy does decimal arithmetic. Money in this system is whole cents on purpose. A decimal rounding helper on a money path is a penny-drift bug waiting for a big enough number.

Why A is second

The W-2 authorities are merged and verified. The engine that produces the actual numbers is the natural next step, and it is where the two traps below get enforced in code rather than just documented.

Why D is third, and what I want to build

You said this, and you were right:

"The verbatim text and plain English explanations are great, but it's hard to digest as there is a wall of words and color."

A wall of quotes is **evidence**. It is not a **worksheet**. Sage shows you the form because *the form is the mental model* — you do not think "box 1 of the W-2," you think *that box, there, top-left*.

So: **three tabs — Form, Why, Check.**

- **Form** — the form laid out the way it is actually printed. Real numbers in the boxes. Boxes that still need something are **red**. And the part I think matters most: boxes that are **correctly empty are grey, with the reason attached** — "blank on purpose: Washington has no income tax." A box that is *supposed* to be blank must never look like a box you forgot. That distinction is the whole game, and box 17 is the worked example.
- **Why** — click a box, get *that box's* law. Same verbatim quotes, three sentences at a time instead of forty. The content does

not change; the *timing* does. You get the answer at the moment you have the question.

- **Check** — the reconciliations. Does the W-3 tie to your four 941s?

Built **once, generically**, so every form inherits it — not rebuilt per form.

Two honest cautions. Reproducing IRS forms exactly is fiddly detail work, so this will take real effort to look right. And this is a **worksheet and review** surface, not a filing surface — anything printed from it gets marked as such. Our boundary has not moved: we replace the data-preparation half of what Aatrix does. We do not become a filing agent.

Why B is last, and it is not a matter of taste

You asked for the K-1, the 1120-S and the 1040, and said something sensible about it: "*We may not have the info yet we need. But we can build the logic and forms themselves to be ready to fill with real data once I have it.*"

Agreed — for the structure. But there is one hard blocker, and it is worth understanding rather than working around.

Form 7203 is your stock and debt basis. Under §1366(d)(1), you may deduct S-corporation losses **only up to your basis**. Not "mostly," not "usually" — that is the ceiling. Without 7203, whether a loss on your personal return is deductible **this year, next year, or never** is unknown.

I can build the forms and the logic around that number. I cannot invent the number. So the plan is: build the structure, and have every unfilled field **refuse** rather than quietly default to zero. A form that shows a confident zero where the truth is "we do not know yet" is worse than a form that stops and tells you what is missing.

Two things about your own W-2 that look like bugs and are not

These are now recorded in the roadmap, in the code, and in tests, because a future maintainer — possibly me, having forgotten — will look at them and try to "fix" them.

1. Your box 1 will be HIGHER than boxes 3 and 5. That is correct.

Greenway pays your health insurance. You own 85% of an S corporation. So:

- **Box 1** (wages for income tax) — the premiums **are** included. The IRS instructions list them explicitly, item 5.
- **Boxes 3 and 5** (Social Security and Medicare wages) — the premiums are **excluded**, by §3121(a)(2)(B).

The result is a W-2 where box 1 exceeds boxes 3 and 5 by exactly the premium. Every instinct says that is a mistake. **It is not.** If someone "corrects" it so the boxes agree, they have either overpaid your FICA or understated your income.

The good news: you then generally deduct that same amount on the front of your 1040 as self-employed health insurance, so the tax usually washes out. **But only if it went on the W-2 first.** Miss it here and you lose the deduction there. That is why this is worth knowing rather than trusting to software.

2. Box 17 must be blank.

Washington has no personal income tax, so there is nothing to report as state income tax withheld.

The trap: your employees *do* have state money withheld — Paid Family & Medical Leave and WA Cares. Those are real. **They are not income tax.** They belong in **box 14** ("Other"). Put them in box 17 and you have told the IRS your employees paid a state income tax that does not exist, and invited them to claim a deduction they are not entitled to.

What I still need from you, and when

Nothing blocks C, A or D. These block **B**:

1. **Form 7203 for each shareholder** — the basis figures. The big one.
2. **Form 2553 and the CP261 acceptance letter** — proof of the S election.
3. **The ending AAA from Schedule M-2**, and the S-election tax year.
4. **The 2027 rates** (twelve of them) — time-critical, because your first payroll is 1 January 2027.
5. **Q3/Q4 2026 filed returns**, the salaried reporting method, the §6699(e) amount, and the quarterly federal short-term rates.

Still outstanding from before, not blocking the next three slices: the depreciation schedule, the work papers, twelve years of returns, and the intercompany detail between the four entities.

Closed since last time: the Wells Fargo loan, via your Plaid connection. Thank you — that removes a real blocker on the balance sheet.

One last thing, on how this is going

You asked me to coach you into being the best accountant you can be. Here is some genuine feedback rather than encouragement.

The two defects I found in my own work this slice were both caught by the same habit, and it is a habit worth stealing:

I compared the new thing to its neighbours. The 28 dead links were not found by a clever test. They were found because six *older* citations pointing at the same IRS document did it differently. When something new sits beside something old that already works, **diff the shapes**. Consistency with a working neighbour is the cheapest oracle in existence, and you can use it before you run anything.

The second one is more uncomfortable. I wrote down an identifier **from memory of what it was about** rather than reading what it was actually called — and it silently cost an authority, with every check still green. That is the same mistake as remembering roughly what a number was instead of looking it up. In accounting it is called tying to source, and the reason it is a discipline rather than a preference is that being *approximately* right feels exactly like being right.

That is why "never guess, never assume, test everything" is the standing instruction, and why I keep reporting my own defects to you instead of quietly fixing them. The value is not that the system is never wrong. It is that when it is wrong, something says so out loud.

Next up: **slice C — turning on 82 lessons you already own.**